

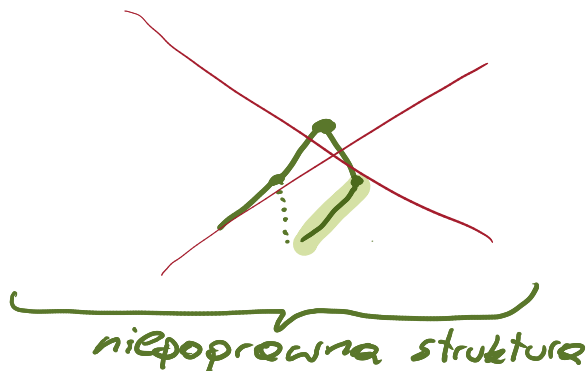
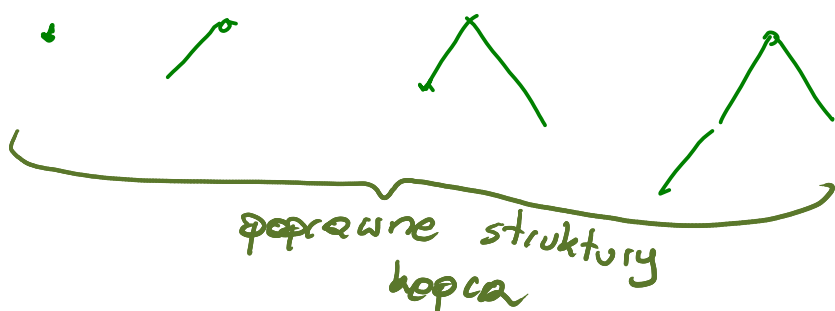
# AlSD

## WYKZAD 11

## Kopiec

**Kopiec** - drzewo binarne spełniające następujące warunki:

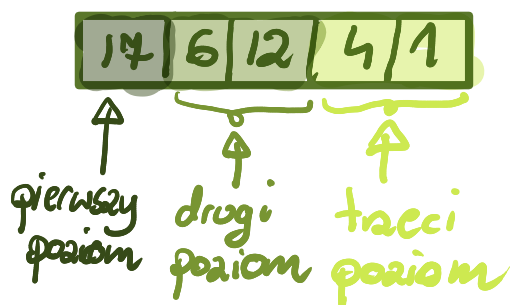
1° liście dobudowujemy w pierwszym możliwym miejscu od lewej



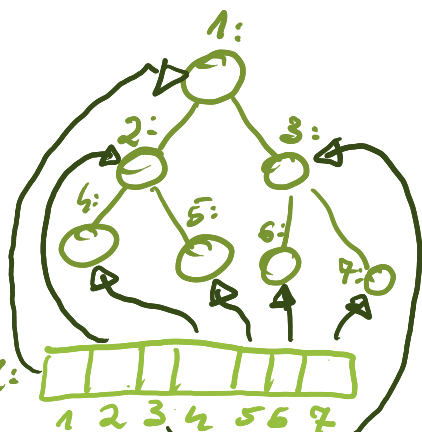
2°  $\forall v \in K \leq \text{ojciec}(v)$ , gdzie  $K$  to kopiec



Pamiętanie kopców w tablicy:



Oczywiście 20 miast  $\leq$  może być inny porządek



| $i$ : | $\text{syn}(i)$ | $\text{ojciec}(i)$ |
|-------|-----------------|--------------------|
| 1     | 2, 3            | —                  |
| 2     | 4, 5            | 1                  |
| 3     | 6, 7            | 1                  |

**Wniosek:**

- synowie  $K[i]$  są w  $K[2i]$  oraz  $K[2i+1]$
- ojciec  $K[i]$  jest w  $K[i \div 2]$

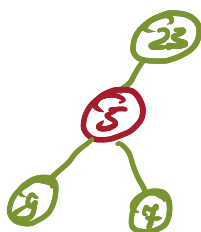
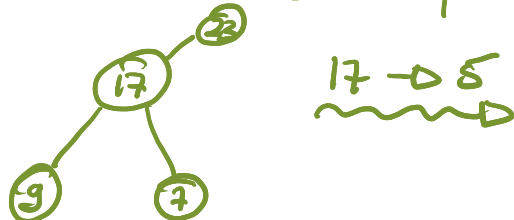
Numery nad węzłami to indeksy, a nie klucze!

# Operacje na kopcach:

→ zmiana wartości w węźle - może zepsuć porządek koprowy - trzeba będzie go naprawić:

1° Porządek niezburzony - zostawiamy

2° Element po zamianie jest mniejszy od dzieci:



Wtedy 5 trzeba przesunąć niżej

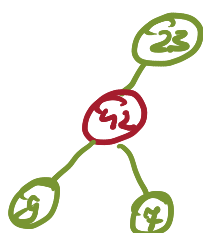
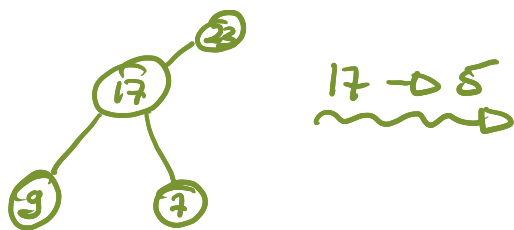
## Idea przesun'-niżej:

- ~ Dopóki nowoustawiona wartość zaburza porządek - swapujemy ją z największym z dzieci.
- ~ One dziecko było przed ustaleniem mniejsze od dziadka, więc po swapie nie będzie od niego większy
- ~ z drugiej strony jest większe od ojca i od brata, więc po swapie będzie większy od dzieci

Pseudokod:

```
przesun'-niżej( $K[1..n]$ ,  $i$ ):  
   $k \leftarrow i$   
  do  $j \neq k$   
    if  $2j \leq n$  and  $K[k] < K[2j]$ :  
       $k \leftarrow 2j$   
    if  $2j < n$  and  $K[k] < K[2j+1]$ :  
       $k \leftarrow 2j+1$   
     $K[k] \leftrightarrow K[j]$   
  while  $j \neq k$ 
```

3° Element po zamianie jest większy:



Wtedy 52 trzeba przesunąć wyżej

## Idea przesun'-wyżej:

- ~ Dopóki nowoustawiona wartość zaburza porządek - swapujemy ją z ojcem

przesun'-wyżej( $K[1..n]$ ,  $i$ ):

```
 $k \leftarrow i$   
do  $j \neq k$   
  if  $j \text{ div } 2 > 0$  and  $K[j] > K[j \text{ div } 2]$ :  
     $k \leftarrow j \text{ div } 2$   
   $K[j] \leftrightarrow K[k]$   
while  $j \neq k$ 
```

Dowód poprawności - zostawiamy czytelnikowi jako ćwiczenie:)

Skomplikacja  $O(\log n)$

# Budowa kopca

Dane:  $a_1, \dots, a_n$  - elementy mające znaleźć się w kopcu.

Najpierw - wstawiamy elementy kolejno, a jeśli zepsuje porządek - przesuwamy wyżej:

{ 4 7 1 5 6 8 11 19 }

① Wstawiamy 4

② Wstawiamy 7

③

⇒

④

⑤ swapujemy 4 i 7

⑥ Wstawiamy 1:

⑦ → psuje porządek  
przesuwamy wyżej

④

⇒

④

⇒ ...

Pseudokod:

buduj-kopiec- $up(a_1, \dots, a_n)$

$K \leftarrow [a_1, \dots, a_n]$

for  $i = 1$  to  $n$ :

przesun-wyzej( $K, i$ )

Dowód poprawności:

Również zostawiam jako ćwiczenie:)

Najprościej → pokazać, że własność

" $K[1..i]$  jest kopcem" jest niezmiennikiem pętli

Złożoność czasu (worst-case):

→ z jednej strony przesun-wyżej w najgorszym wypadku działa w  $O(\log n)$ , a że wykonujemy je  $n$  razy, każde wywołanie będzie wymagać  $\leq \log n$  porównań → złożoność  $O(n \log n)$

→ z drugiej strony na najniższym poziomie jest blisko połowa elementów (z zależności  $\sum_{i=0}^h 2^i = 2^{h+1} - 1$ ) dla każdego z nich przesun-wyżej będzie wymagać  $\geq \log n$  porównań, czyli algorytm będzie wymagać  $\geq \frac{n}{2} \log(n) = \Omega(n \log n)$ .

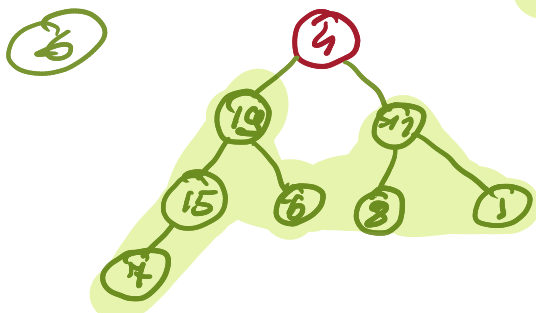
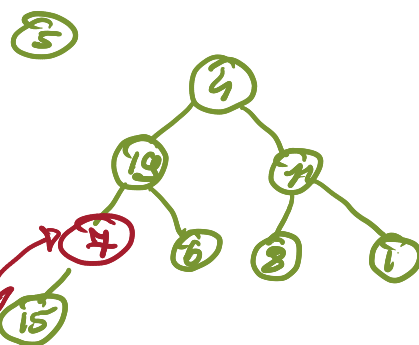
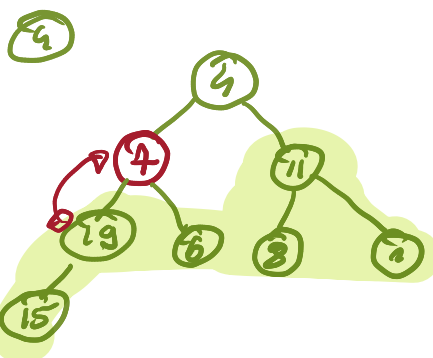
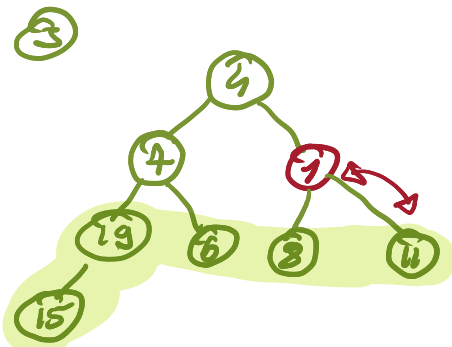
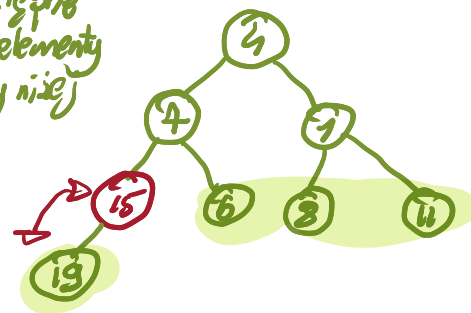
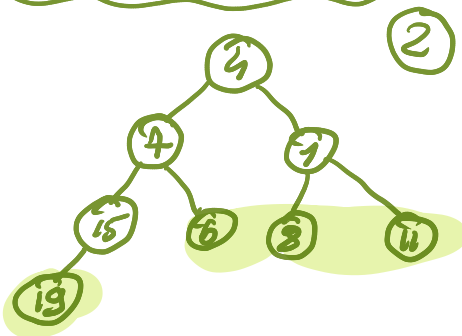
Stąd TC to  $\Theta(n \log n)$

Lepsze podejście - kopiec 1 elementowy jest poprawny, potem ostatnie  $\frac{n}{2}$  elementów to poprawne 1-elementowe kopce, chcemy je łączyć dodając im ojców z dolnej połowy, to może psuć porządek - dlatego ten dodany ojciec będziemy przesuwać niżej:

Przykład:

4 7 1 15 6 8 11 19

1) na i-owym elemencie poprawne kopce  $\rightarrow$



$\Rightarrow$  Przesunięcie 4 w dół pozostawiamy jako ćwiczenie :-)

Pseudokod:

budy-kopiec-v2( $a_1, \dots, a_n$ )  
 $K \leftarrow [a_1, \dots, a_n]$

for  $i = n \div 2$  downto 1:  
 przesun-niziej( $K, i$ )

Dowód poprawności:

Ćwiczenie :-)  
 Najlepiej pokazać, że "kl" jest korzeniem poprawnego kopca" jest niezmiennikiem petli.

Złożoność czasowa:

TW: budy-kopiec-v2 działa w  $O(n)$ .

d-d:

Dowód pokażemy dla  $n = 2^k - 1$  (tj. dla pełnych kopców).

2 razy, że czas rośnie monotonicznie wraz ze wzrostem ilości danych do skopcowania, wyznoszącym 90 z 3 ręgów.

Niech  $h$  (wysokość) oznacza długość najdłuższej ścieżki do liścia z danego wierzchołka.



Z reguły, że kopiec jest pełny to dla każdego poziomu  $h$  będzie stałe.

Zauważmy, że wobec, że taki kopiec, którego korzeń ma wysokość  $h$  ma  $2^{h+1}-1$  wierzchołków

(dla  $h=0$  mamy  $1 = 2^1-1$ , dla  $h>0$  mamy

2 pełne podkopcze o wysokości  $h-1$ , czyli indukcyjnie

po  $2^h-1$  wierzchołków, wtedy to o wysokości  $h$  ma

1 wierzchołek z korzeniem + wierzchołki z podkopców, czyli  $2(2^h-1)+1 = 2^{h+1}-1$  wierzchołków). <sup>indukcja od kor.:)</sup>

Skoro tak, to wierzchołków o wysokości  $h$  jest

co najwyżej  $\lceil \frac{n}{2^{h+1}-1} \rceil$  (gdy rozdzielimy wszystkie

$n$  wierzchołków do podkopców o wysokości  $h$  -

intuicja: mamy doniczki, które pomieszczą 10 kwiatów - ile doniczek potrzebujemy

do zasadzenia 42 kwiatów, jeśli każdą będziemy maksymalnie obsadzać?  $\lceil \frac{42}{10} \rceil = 5$ ). Ale

$$2^h - 1 \geq 0 \quad 1 + 2^h$$

$$2^{h+1} - 1 \geq 2^h, \text{ więc}$$

$$\lceil \frac{n}{2^{h+1}-1} \rceil \leq \lceil \frac{n}{2^h} \rceil.$$

Teraz zauważmy, że jeżeli quścimy przesunąć-niżej na wierzchołku o wysokości  $h$ , to przemiecy on cały ścieżkę do liścia, zatem wykonamy  $\leq h$  przesunięć.

Stąd sumarycznie ( $T(i)$  oznacza czas działania przesunąć-niżej

dla  $i$ -tego wierzchołka):  
 $\sum_{i=1}^n T(i) \leq \sum_{h=0}^{\log n} \lceil \frac{n}{2^h} \rceil \cdot h$   
maksymalna wysokość    ile wierzchołków o wys.  $h$     ile przesunięć wymaga przesunąć-niżej

$$(1) \leq 2 \cdot \sum_{h=0}^{\lceil \log n \rceil} \frac{n}{2^h} \cdot h \leq n \cdot \sum_{h=1}^{\infty} \frac{h}{2^{h-1}} \quad (2)$$

(1)  $\lceil \frac{n}{2^h} \rceil \leq 2 \cdot \frac{n}{2^h}$ , ponieważ  $2^h \leq n$  - jest tak,  
bo kopiec o wysokości  $h$ , ma  $2^{h+1} - 1$  wierzchołków,  
więc jeżeli  $n$  to liczba wierzchołków takiego kopca, to  
 $n \geq \underbrace{2^{h+1} - 1}_{\text{to pokazaliśmy już}} \geq 2^h$ , zatem  $\frac{n}{2^h} \geq 1$ , a stąd

części całkowite  $\lceil \frac{n}{2^h} \rceil \geq 1$ , a że  $\lceil \frac{n}{2^h} \rceil < \frac{n}{2^h} + 1$ ,  
to  $2 \cdot \frac{n}{2^h} = \frac{n}{2^h} + \frac{n}{2^h} \geq \frac{n}{2^h} + \lceil \frac{n}{2^h} \rceil \geq \frac{n}{2^h} + 1 \geq \lceil \frac{n}{2^h} \rceil$

(2) Ile wynosi ten szereg?

a MOK:

$$\sum_{i=0}^{\infty} r^i = \frac{1}{1-r} \quad r = \frac{1}{2} \Rightarrow \sum_{h=1}^{\infty} \frac{h}{2^h} = \frac{1}{(1-\frac{1}{2})^2} = 4$$

$$\left( \sum_{i=0}^{\infty} i r^{i-1} \right)' = \frac{1}{(1-r)^2}$$

Stąd  $(2) \approx n \cdot 4 = O(n)$

Mam wrażenie, że przekombinowałem (intuicja z doniczkami:  
NAPRAWDĘ BYŁA MI POTRZEBNA), noży się, jak znaleźć  
prostsze rozumowanie, to dowód poprawię :-)

\* „doniczki” wróć po UNION-FINDZIE :-)

Ograniczenie od dołu jest

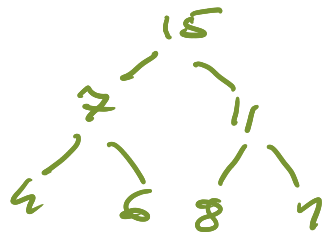
oczywiste - główny for w buduj-kopiec-  
v2 wykonuje  $\lceil \frac{n}{2} \rceil$  obrotów, więc  
mamy ich  $\Omega(\frac{n}{2})$ . Stąd  
złożoność buduj-kopiec-v2 to  $\Omega(n)$ .

Dla innych  $n$ :

$$\begin{aligned} 2^k - 1 &\leq n \leq 2^{k+1} - 1 \\ \Theta(2^k) &\leq n \leq \Theta(2^{k+1}) \\ \Theta(2^k) &\leq n \leq \Theta(2^k) \\ &\Rightarrow \Theta(n) \end{aligned}$$

# Zastosowania

## 1<sup>o</sup> Heapsort:

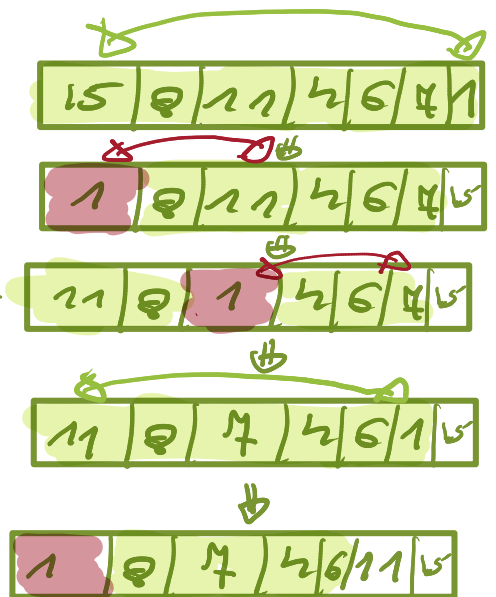


1. Zbuduj kopiec  $\Theta(n)$
2. Dopóki kopiec nie pusty
  - ~ Usuwamy maksimum
  - ~ wstawiamy ostatni element do korzenia
  - ~ przystrojenie struktury kopca  $\Theta(n \log n)$

pseudokod:

```

heapsort(K[1...n]):
    buduj-kopiec(K[1...n])
    for i = n downto 2:
        K[i] ↔ K[1]
        przesun-nizej(K[1...i], 1)
    
```



na jasnozielono - rozmiarowy obszar kopca po heapSort  
 na ciemnoniebiesko - przesuwany element  
 na białym - elementy posortowane (tj. usunięte z kopca)

Poprawność - również pozostałom czynnikiem. Tym razem niezmiennik fore również pozostałom do odgadnięcia: -)

## Modyfikacje:

- przesuwamy luke go najniższym elemencie na sam dół kopca, wstawiamy element i go wstawiamy jeśli porządek jest zachowany - asymptotycznie to nic nie da, ale statystycznie już tak

Własności:

- ~ Optymistyczny czas działania  $\Theta(n)$
- ~ Sortuje w miejscu (tylko wyjściową tablicę przekształca w kopiec)
- ~ jest niestabilny, przykład:

WE: 

|                |                |                |
|----------------|----------------|----------------|
| 1 <sub>1</sub> | 1 <sub>2</sub> | 1 <sub>3</sub> |
|----------------|----------------|----------------|

 → jest dobrym kopcem (dla indeks - kolejność występowania w wyjściowej tablicy).

Heapsort:



Czyli kolejność identycznych elementów uległa zmianie :)

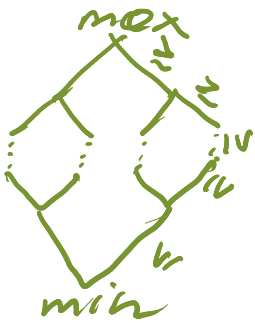
QuickSort jest lepszy.

## 2<sup>o</sup> Rolejka priorytetowa:

## Operacije:

$\sim$  insect (minimum)  $(\leq)$   
 $\sim$  sprouts maximum (4 gozdotu  $\geq$ )  
 $\sim$  uson minimum  $\sim$   $(- \infty)$

## Kolejka podwójna (kopiec minimeksowy)

 $Q_1 \dots Q_n$ 

→ dwa kopce - porodek jest też na krańcach pomiędzy kopcami.

TODD - notatka o tych koźcach jako portafila  
z ćwiczeń i opisać powyższy sposób, bo ten z wikipedii  
jest toporny i na egzaminie przepolicas :-),

Polować się u adwersara i wymyślić pytania do egzaminu :-)

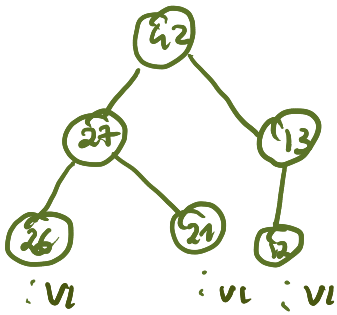
# Kopiec minmaksowy

Yoda:

Jakiś pseudokod.

Ze dużo czasu straciłem  
na rekminowanie wzorów  
na ojce i syny :-)

H:



H:

42 | 27 | 13 | 26 | 21 | 12

L:

3 | 4 | 9 | 11 | 15 | 7

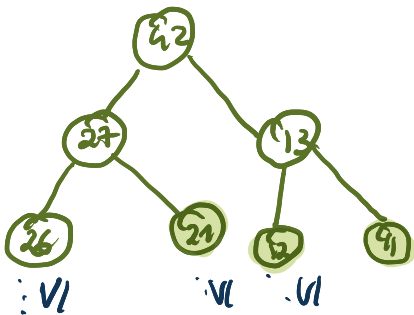
liście

$$|H| = \lceil \frac{n}{2} \rceil$$

$$|H| + |L| = n$$

$$|L| = \lfloor \frac{n}{2} \rfloor$$

H:



H:

42 | 27 | 13 | 26 | 21 | 12 | 11

L:

3 | 4 | 5 | 14 | 6 | 7

liście

$$|H| = \lceil \frac{n}{2} \rceil$$

$$|H| + |L| = n$$

$$|L| = \lfloor \frac{n}{2} \rfloor$$

Idea

42 | 3 | 27 | 5 | 13 | 4 | 26 | 14 | 21 | 6 | 12 | 7 | 11

Trzymamy kopce w jednej tablicy tak, aby nie przemienić  
ustępowość z jednego i drugiego kopca.

Chcemy, aby  $|L| \leq |H| \leq |L| + 1$ , zatem

Elementy z H są na nieparzystych pozycjach, a L  
na parzystych - wtedy gdy n nieparzyste  $|H| = |L| + 1$ , wpp.  
 $|L| = |H|$

Problem:

Jak otagmywać relacje  $Ojciec(i)$ ,  $Syn(i)$

1° Grupujemy w komórki po 2, wtedy  $i$  znajduje się w komórce  $i' = \lceil \frac{i}{2} \rceil = i \text{ div } 2 + i \bmod 2$ .

2° Wtedy dzieci  $i$  znajdują się w komórkach  $k_l' = 2i'$  oraz  $k_r' = 2i' + 1$  na polach 1 (gdzie  $i \neq 1$ ) lub 2 (gdzie  $i = 1$ ) komórek  $k_l', k_r'$ .

3° Pole 1  $k_l' \rightarrow 2k_l' - 1$  Komórka 2  $\rightarrow 2k_l'$

Zatem:

$$\text{synlew}(i) = 2k_l' - i \bmod 2 = 4i' - i \bmod 2 = 4(i \text{ div } 2 + i \bmod 2) - i \bmod 2$$

$$= 4i \text{ div } 2 + 3i \bmod 2$$

$$\text{synPraw}(i) = 2k_r' - i \bmod 2 = 2(2i' + 1) - i \bmod 2 = 4i' + 2 - i \bmod 2 =$$

$$= 4i \text{ div } 2 + 3i \bmod 2 + 2 = \text{synlew}(i) + 2$$

|      |    |   |    |   |    |   |    |
|------|----|---|----|---|----|---|----|
| $i'$ | 1  | 2 | 3  | 4 | 5  | 6 | 7  |
|      | 42 | 3 | 27 | 5 | 13 | 4 | 26 |
| $i$  | 1  | 2 | 3  | 4 | 5  | 6 | 7  |

nr komórki gdzie jest tato

$$\text{Ojciec}(i) = 2 \left( (i \text{ div } 2 + i \bmod 2) \text{ div } 2 \right) - i \bmod 2$$

nr komórki, gdzie jest  $i$

$$i' = i \text{ div } 2 + i \bmod 2$$

$$i' - i \bmod 2 = i \text{ div } 2$$

$$i' - i \bmod 2 = \frac{i - i \bmod 2}{2}$$

$$2i' - 2i \bmod 2 = i - i \bmod 2$$

$$2i' - i \bmod 2 = i$$

znany, bo wemy, jak jest parzystość syna



Chcemy teraz rozwiązać logikę przechodzenia między H i L.

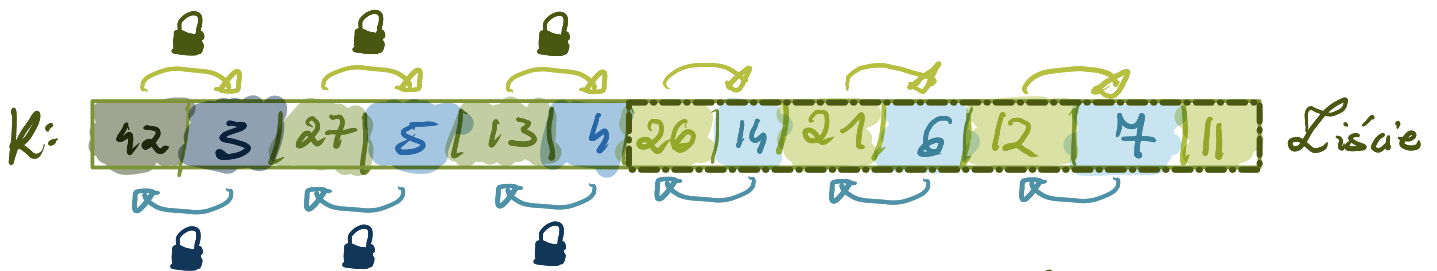
Pomysł 1 (raczej słaby)

Tworzymy połączenia między  $K[i]$  oraz  $K[i+1]$ , ale z reguły, że chcemy je mieć tylko pomiędzy liśćmi na pozostałe inne następujące kłódki.

Jeżeli  $K[2j-1]$  oraz  $K[2j]$  są liśćmi, to chcemy mieć warunek  $K[2j-1] \geq K[2j]$ .

Wtedy  $\text{Następnik}(i) = L \cdot i + (-1)^{i \bmod 2}$ .

Do sprawdzenia czy  $i \rightarrow \text{Następnik}(i)$  nie jest zablokowane wystarczy stwierdzić, czy mają synów.



Wtedy zmiana wartości elementu (np. do górnego końca) - polegałaby na przesunięciu w dół końca ustalonego elementu sprawdzenie następnika, a potem ewentualnie wywindowanie wyżej w drugim końcu

usunięcie minimum/maksimum - tutaj jakiś kłódeczko może się odblokowac, więc trzeba zewyflować, czy jej odblokowanie nie zabuży ogółku, potem standardowo zamieniamy minimum/maksimum z n-tym element i przesuwamy dalej.

Pomysł 2 (raczej lepszy)

Zamiast kłódeczek mamy globalny niezmiennik  $K[2j-1] \geq K[2j]$ . Da się pokazać, że przyczeranie ogółku można zrobić przesuwając tylko 2 elementy - 1 z H drugi z L.



# Kopiec minimaksowy

- ~ KŁO sugerował symetryczny kopiec minimaksowy opisany w pewnym artykule (odnośnik [1] u notatce o kopcach) przez A. Arvind, C. Gordon Rongen, więc się nim posługuję. Ten kopiec nazywa się SMM.

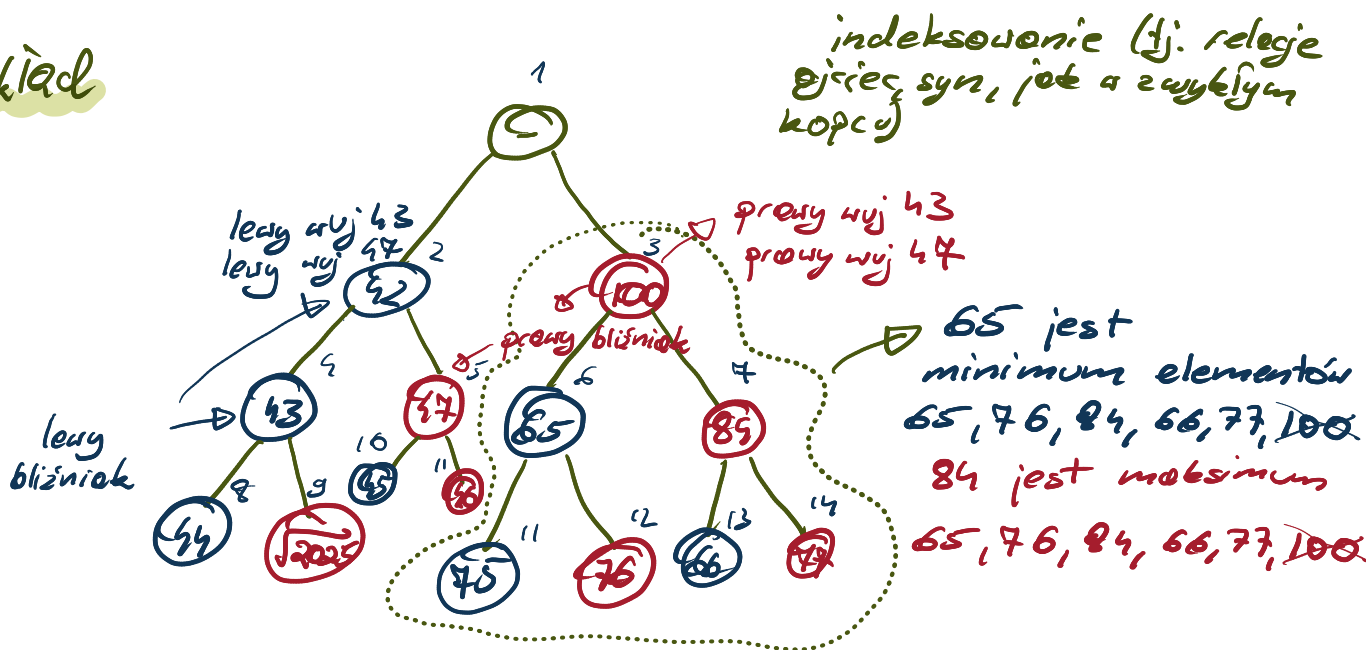
## Idea

- ~ Root jest pustym wierzchołkiem
- ~  $\forall v$  lewe dziecko  $v$  jest minimalne w  $T_v \setminus \{v\}$  (w poddrzewie o korzeniu  $v$ ), a prawe maksymalne w  $T_v \setminus \{v\}$

## Definicje:

- ~ prawy, lewy bliźniak ~ prawy i lewy syn tego samego korzenia
- ~ niedł  $x$  i  $y$  będą kolejno lewym i prawym bliźniakiem, a z będzie potomkiem  $x$  albo  $y$ , wtedy  $x$  jest lewym wujem  $z$ , a  $y$  prawym.

## Przykład



**Lemat** Kopiec  $H$  jest SMM wtw:

- (1)  $\forall v \in H$   $\text{lewyWuj}(v) \leq v \wedge \text{prawyWuj}(v) \geq v$
- (2) wartość lewego bliźniaka jest mniejsza niż prawego.

d-d Pokazemy że dla „minimum” dla maksimum jest analogicznie

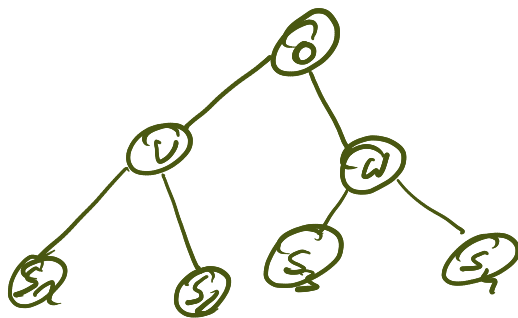
$\Rightarrow$  (1) Niech  $v$  ma lewego ojca  $w$ , a  $u$  ma ojca  $u$ . Wówczas  $w$  jest minimum w  $T_0 \setminus \{u\}$ .

$$v \in T_0 \setminus \{u\} \Rightarrow v \leq \min(T_0 \setminus \{u\}) = w$$

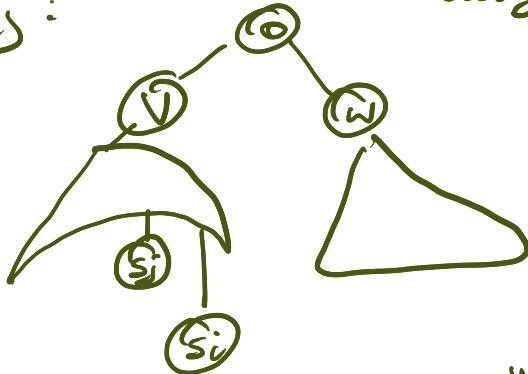
(2) Niech  $o$  jest ojcem  $v$ . Niech  $v$  będzie lewym bliźniakiem, a  $u$  prawym. Wówczas  $o$  jest maksimum

w  $T_0 \setminus \{u\}$ , więc  $v \leq u$  (bo  $v \in T_0 \setminus \{u\}$ )

$\Leftarrow$  Weźmy dowolne  $v$ . Przeprowadzimy indukcję po poziomach w dół od  $v$ .



Dołącz:



$v$  jest lewym dzieckiem  $o$ ,  
zatem dla  $s_1, s_2, s_3, s_4$   
 $v$  jest lewym wujkiem, więc mamy

$s_i \leq v$   
Ponadto relacji  $v \leq w$  a więc bliźniaczej

$s_j$  - dowolny lewy  
wujek  $s_j^*$

$$s_i \geq s_j \geq v$$

\*  $s_j^*$  istnieje bo  $v$  jest lewym,  
więc poprzedni poziom został wypełniony.

Insert( $i, K[1 \dots n]$ ): (wg przytoczonego artykułu)

$K[n+1] \leftarrow i$

if  $i$  jest prawym bliźniakiem  $j$  oraz  $j > i$

swap( $K[i], K[j]$ )

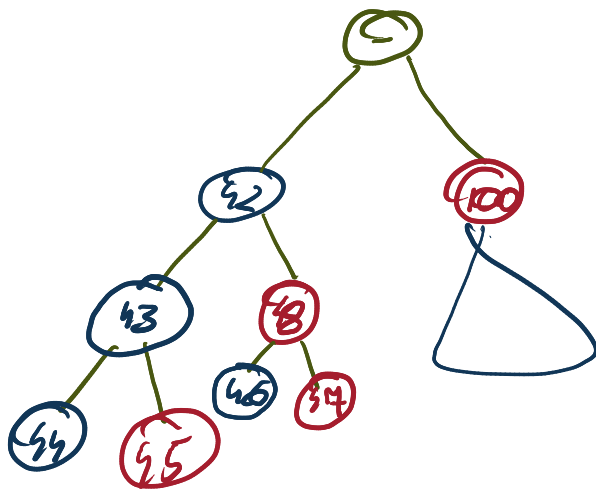
stop & false

while  $\neg$  stop

if  $K[i] < K[\text{lewyWuj}(i)]$  swap( $K[i], K[\text{lewyWuj}(i)]$ )

else if  $K[i] > K[\text{prawyWuj}(i)]$  - analogicznie

else stop & true



Przypuśćmy, że w miejsce 43 wstawimy nowy element  $v$ , co się może zepsuć?

- ①  $v$  jest mniejsze od lewego wujka 42
- ②  $v$  jest większe od prawego bliźniaka 47
- ③  $v$  jest większe od prawego wujka 100
- ④  $v$  jest większe od jakiegoś z dzieci (44, 45) i „bratanków” (46, 47), tzn. tych wnuczków dla których jest lewym wujkiem.

⑤  $\Rightarrow$  można swapować z minimum (lewych bliźniaków)  
 $\Rightarrow$  przesunąć niżej w końcu min

①, ②, ③  $\Rightarrow$  potrz: insert.

Pseudokod:

przesun'-wyżej-po-maksimach ( $K[1 \dots n]$ ,  $i$ ):

//zobędziemy, że ① jest naszym zmartwieciem  
 prawy-wujek =  $\frac{i \text{ div } 2 + (i+1) \bmod 2}{\text{ojciec}}$

jeśli ojciec był lewym wujkiem, chcemy jego bliźniaka

if ( $K[\text{prawy-wujek}] < K[i]$ ):

$K[i] \leftrightarrow K[\text{prawy-wujek}]$

przesun'-wyżej-po-maksimach ( $K[1 \dots n]$ ,  $i$ ):

Poprawności ~ przed ustąpieniem  $K[i]$   $K[\text{prawy\_wujek}]$  był maksimum a suym poddzwie, więc a szczególności był maksimum a poddzwie  $K[i]$ .

$K[i] > K[\text{prawy\_wujek}]$ , więc  $K[i]$  jest maksymalny a poddzwie  $K[\text{prawy\_wujek}]$ .

przesun' - niżej - po - minimumach ( $K[1...n]$ )

// patamy na prawego bliźniaka

if ( $K[i] > K[i+1]$ ):

$K[i] \leftrightarrow K[i+1]$

przesun' - wyżej - po - maksimumach ( $K[1...n], i+1$ )

// być może  $K[i+1]$  jest teraz większe od prawego wujka

// więc przesuwamy go (uwaga - to wykonamy tylko raz!)

// określamy, że  $K[i] \geq$  lewego wujka

j = taki indeks, że  $K[j] = \max(K[2i], K[2i+1])$

$K[j] \leftrightarrow K[i]$  // ustelamy nowe minimum

przesun' - niżej - po - minimumach ( $K[1...n], i$ )

Ostatecznie:

przesun' - niżej ( $K[1...n], i$ ):

if i parzyste:

if  $K[i] > K[i+1] \vee K[i] > K[2i] \vee K[i] > K[2i+1]$

przesun' - niżej - po minimumach ( $K[1...n], i$ )

if i nieparzyste:

// analogicznie, ale z odpowiednim porządkiem

heapify( $K[1...n]$ ):

for  $i = n \text{ div } 2$  down to 1:

przesun' - niżej ( $K[1...n], i$ )

delete-min ( $K[1...n], i$ ):

$K[2] \leftrightarrow K[n]$

przesun' - niżej ( $K[1...n-1], 2$ )

delete-max // analogicznie ale z 3, a nie 2

